```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score, f1_score

df = pd.read_csv('/content/CustomerChurn_Preprocessed.csv')

print("Dataset Shape:", df.shape)
print("\nColumns:")
print(df.columns.tolist())
print("\nFirst 5 Rows:")
display(df.head())

target_candidates = ['Churn', 'churn', 'Customer_Status', 'customer_status', 'Exited', 'exited']

target = None

for col in target_candidates:
    if col in df.columns:
        target = col
        break

if target is None:
    target = df.columns[-1]

print("\nTarget Column:", target)

X = df.drop(columns=[target])
y = df[target]

for col in X.columns:
    if X[col].dtype == 'object':
        le = LabelEncoder()
        X[col] = le.fit_transform(X[col].astype(str))

if y.dtype == 'object':
    y = y.astype(str).str.strip().str.lower()
    y = y.map({
        'yes': 1,
        'no': 0,
        'true': 1,
        'false': 0,
        'churn': 1,
        'not churn': 0,
        'churned': 1,
        'active': 0
    })
```

```
    })

    if y.isna().any():
        le = LabelEncoder()
        y = le.fit_transform(y.astype(str))

X = X.apply(pd.to_numeric, errors='coerce')
X = X.fillna(X.median(numeric_only=True))
X = X.fillna(0)

y = pd.to_numeric(y, errors='coerce')
valid = y.notna()

X = X.loc[valid]
y = y.loc[valid]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

logistic_model = LogisticRegression(max_iter=1000)
decision_tree_model = DecisionTreeClassifier(random_state=42)
random_forest_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

logistic_model.fit(X_train_scaled, y_train)
decision_tree_model.fit(X_train, y_train)
random_forest_model.fit(X_train, y_train)

logistic_pred = logistic_model.predict(X_test_scaled)
decision_tree_pred = decision_tree_model.predict(X_test)
random_forest_pred = random_forest_model.predict(X_test)

models = {
    'Logistic Regression': logistic_pred,
    'Decision Tree': decision_tree_pred,
    'Random Forest': random_forest_pred
}

results = []

for name, prediction in models.items():
    accuracy = accuracy_score(y_test, prediction)
    precision = precision_score(y_test, prediction, zero_division=0)
    recall = recall_score(y_test, prediction, zero_division=0)
    f1 = f1_score(y_test, prediction, zero_division=0)
```

```
    results.append([
        name,
        accuracy,
        precision,
        recall,
        f1
    ])

results_df = pd.DataFrame(
    results,
    columns=[
        'Model',
        'Accuracy',
        'Precision',
        'Recall',
        'F1-Score'
    ]
)

print("\nModel Performance:")
display(results_df)

for name, prediction in models.items():
    cm = confusion_matrix(y_test, prediction)

    plt.figure(figsize=(5, 4))
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        xticklabels=['No Churn', 'Churn'],
        yticklabels=['No Churn', 'Churn']
    )
    plt.title(name + ' Confusion Matrix')
    plt.xlabel('Predicted')
    plt.ylabel('Actual')
    plt.show()

metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']

results_df.set_index('Model')[metrics].plot(
    kind='bar',
    figsize=(10, 6)
)

plt.title('Classification Model Performance Comparison')
plt.xlabel('Models')
plt.ylabel('Score')
plt.ylim(0, 1)
plt.xticks(rotation=0)
plt.legend(title='Metrics')
plt.grid(axis='y')
plt.show()

best_model = results_df.loc[
```

```
    results_df['F1-Score'].idxmax(),
    'Model'
]

print("\nBest Performing Model:", best_model)

if best_model == 'Logistic Regression':
    importance = pd.Series(
        np.abs(logistic_model.coef_[0]),
        index=X.columns
    ).sort_values(ascending=False)
elif best_model == 'Decision Tree':
    importance = pd.Series(
        decision_tree_model.feature_importances_,
        index=X.columns
    ).sort_values(ascending=False)
else:
    importance = pd.Series(
        random_forest_model.feature_importances_,
        index=X.columns
    ).sort_values(ascending=False)

print("\nImportant Factors Influencing Customer Churn:")
print(importance.head(10))

plt.figure(figsize=(10, 6))
importance.head(10).sort_values().plot(kind='barh')
plt.title('Top Factors Influencing Customer Churn')
plt.xlabel('Importance')
plt.ylabel('Features')
plt.show()

print("\nBusiness Recommendations:")
print("1. Identify high-risk customers using the best-performing churn prediction model and provide targeted retention offers.")
print("2. Improve customer service and proactively address issues for customers with high churn probability.")
print("3. Provide personalized discounts, loyalty benefits, and suitable service plans to customers at high risk of churn.")
```

Dataset Shape: (46, 7073)

First 5 Rows:

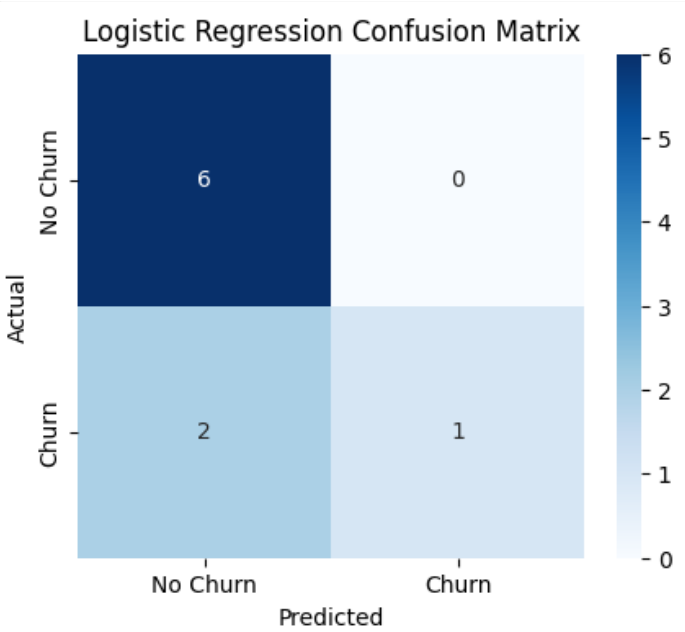
| | SeniorCitizen | tenure | MonthlyCharges | TotalCharges | customerID_0003-MKNFE | customerID_0004-TLHLJ | customerID_0011-IGKFF | customerID_0013-EXCHZ | customerID_0013-MHZWF | customerID_0013-SMEOE | ... | StreamingTV_Yes | StreamingMo internet |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 1 | 29.85 | 29.85 | False | False | False | False | False | False | ... | False | |
| 1 | 0 | 34 | 56.95 | 1889.50 | False | False | False | False | False | False | ... | False | |
| 2 | 0 | 2 | 53.85 | 108.15 | False | False | False | False | False | False | ... | False | |
| 3 | 0 | 45 | 42.30 | 1840.75 | False | False | False | False | False | False | ... | False | |
| 4 | 0 | 2 | 70.70 | 151.65 | False | False | False | False | False | False | ... | False | |

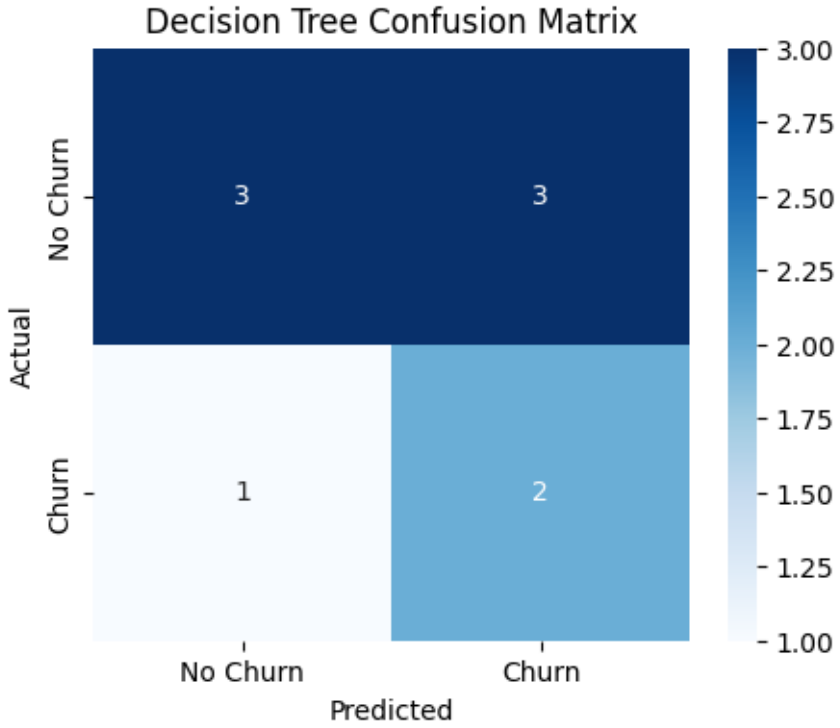
5 rows × 7073 columns

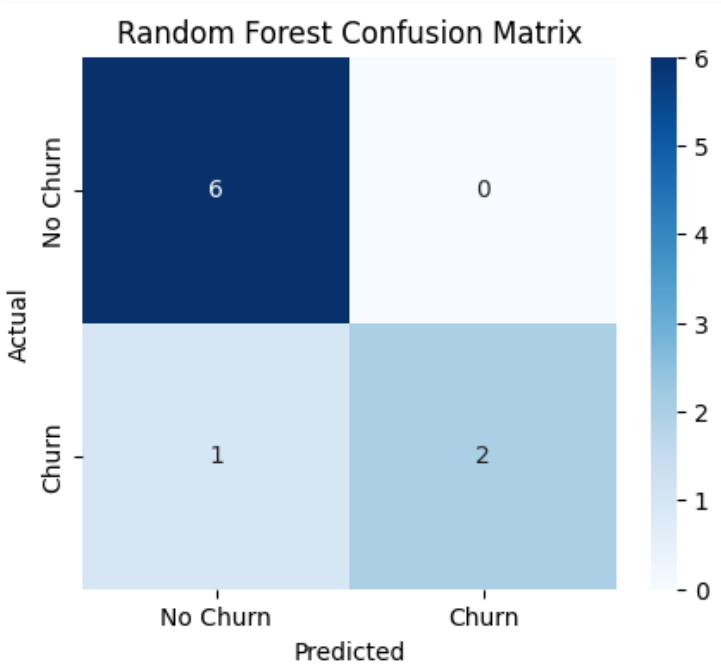
Target Column: Churn

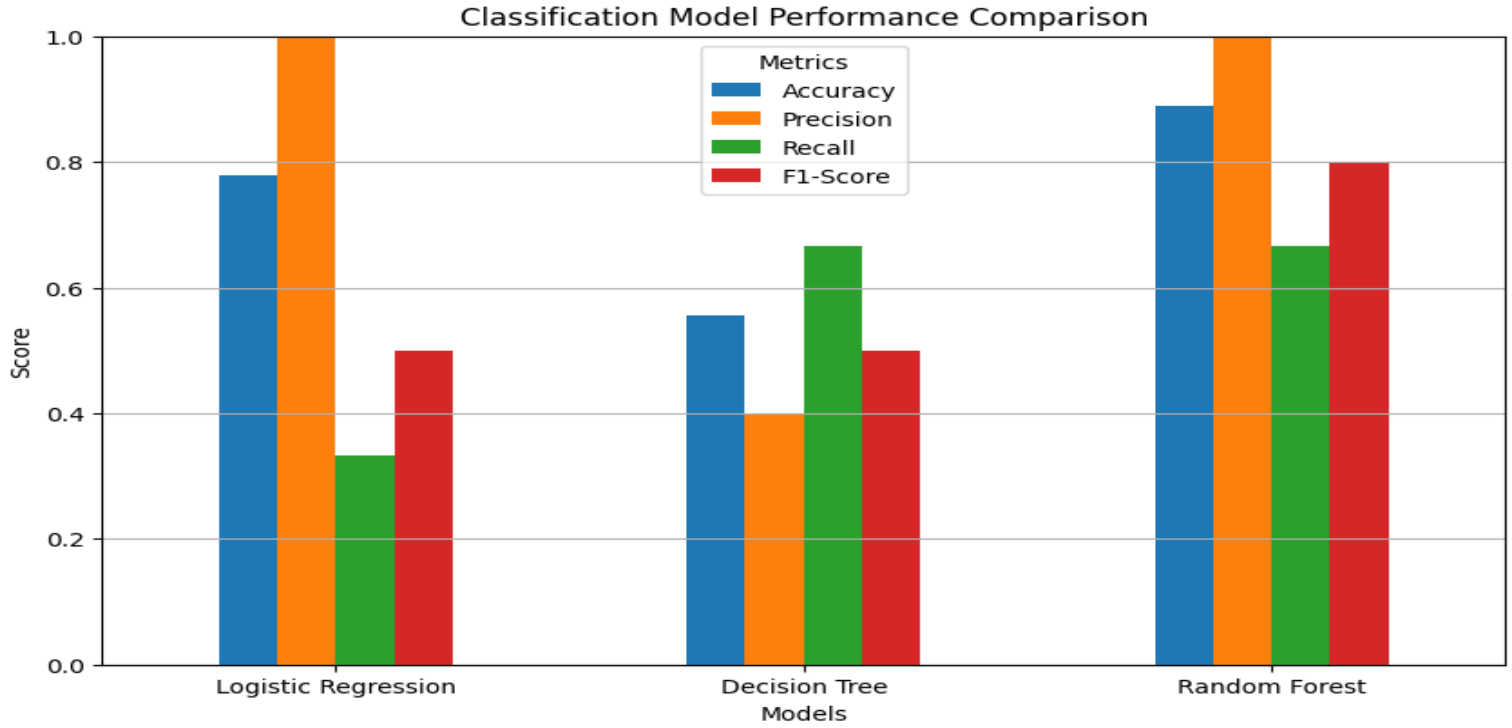
Model Performance:

| | Model | Accuracy | Precision | Recall | F1-Score |
|---|---|---|---|---|---|
| 0 | Logistic Regression | 0.777778 | 1.0 | 0.333333 | 0.5 |
| 1 | Decision Tree | 0.555556 | 0.4 | 0.666667 | 0.5 |
| 2 | Random Forest | 0.888889 | 1.0 | 0.666667 | 0.8 |


Logistic Regression Confusion Matrix
No Churn
Churn
Actual
6
0
2
1
No Churn
Churn
Predicted
6
5
4
3
2
1
0

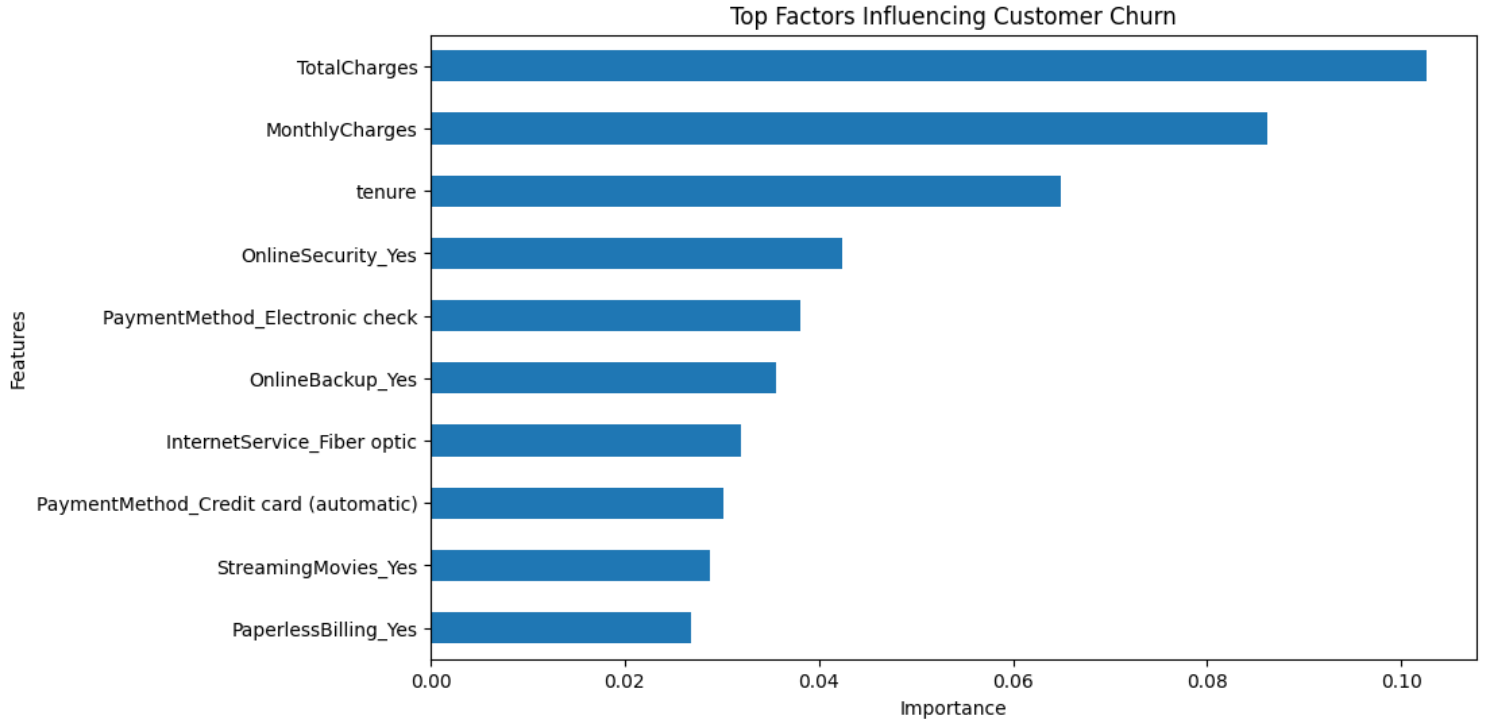

Decision Tree Confusion Matrix
No Churn
Churn
Actual
3
3
1
2
No Churn
Churn
Predicted
3.00
2.75
2.50
2.25
2.00
1.75
1.50
1.25
1.00


Random Forest Confusion Matrix
No Churn
Churn
Actual
6
0
1
2
No Churn
Churn
Predicted
6
5
4
3
2
1
0


Classification Model Performance Comparison
Metrics
Accuracy
Precision
Recall
F1-Score
Score
1.0
0.8
0.6
0.4
0.2
0.0
Logistic Regression
Decision Tree
Random Forest
Models

Best Performing Model: Random Forest

Important Factors Influencing Customer Churn:
TotalCharges 0.102636
MonthlyCharges 0.086148
tenure 0.064987
OnlineSecurity_Yes 0.042465
PaymentMethod_Electronic check 0.038139
OnlineBackup_Yes 0.035633
InternetService_Fiber optic 0.031942
PaymentMethod_Credit card (automatic) 0.030186
StreamingMovies_Yes 0.028723
PaperlessBilling_Yes 0.026885
dtype: float64




Business Recommendations:
1. Identify high-risk customers using the best-performing churn prediction model and provide targeted retention offers.
2. Improve customer service and proactively address issues for customers with high churn probability.
3. Provide personalized discounts, loyalty benefits, and suitable service plans to customers at high risk of churn.